Image Inpainting and Object Removal

Week 2: data pipeline, Telea baseline and first LaMa experiments

Ivan, Alexander, Egor, Tatiana | Computer Vision 2026 | October 13, 2026

Abstract

We compare classical, CNN-based, Transformer-based and diffusion-based inpainting methods for object removal on the CORNE-Val benchmark. This report covers the second week: the implemented data-loading and preprocessing pipeline, the first working baseline (OpenCV Telea) with its initial quantitative results, and the first experiments with pretrained LaMa. LaMa is running end to end, but its current output is unsatisfactory: the object is blurred rather than removed. We describe the observations so far and the experiments planned to find the cause.

Contents

1 Data Loading and Preprocessing **1**
- 1.1 Dataset . . . 2
- 1.2 Download and conversion . . . 2
- 1.3 Loading and preprocessing . . . 2
- 1.4 Mask experiments . . . 2
- 1.5 Metrics and outputs . . . 2

2 Baseline Model: OpenCV Telea **3**
- 2.1 Configuration . . . 3
- 2.2 Training setup . . . 3
- 2.3 Initial quantitative results . . . 3

3 First Experiments with Pretrained LaMa **4**
- 3.1 Current observations . . . 4
- 3.2 Hypotheses and planned experiments . . . 4

4 Problems and Observations **4**

5 Next Steps **5**

6 Code Repository **5**

1 Data Loading and Preprocessing

1.1 Dataset

All quantitative experiments use CORNE-Val as a fixed held-out benchmark. No model is trained or fine-tuned, so no train/validation split is introduced. Every sample provides four images:

- `shot` — the image containing the object to remove (model input);
- `bg` — the object-free background, used as ground truth;
- `mask_sam` — the object-core mask;
- `mask_eff` — the effect-aware mask that also covers shadows and other remnants.

Because `bg` is a real clean background, the evaluation measures actual object removal rather than similarity to an image that still contains the object.

1.2 Download and conversion

The dataset is downloaded from the Hugging Face Hub (`QinmingZhou/CORNE-Val`) with `snapshot_download`. A small script (`data/dataset.py`) matches files across the four folders (`shot/`, `bg/`, `mask_sam/`, `mask_eff/`) by file name and copies them into one folder per sample:

`data/corne-val/<sample_id>/{shot, bg, mask_sam, mask_eff}.png`

1.3 Loading and preprocessing

`benchmark.py` loads every sample folder into memory:

- images are converted to RGB `uint8` arrays;
- masks are converted to single-channel grayscale and binarised with the rule "non-zero means fill";
- images keep their original resolution, so no resizing is applied by the pipeline itself.

Model-specific preprocessing lives inside each model wrapper, so the benchmark treats all models identically. Telea converts RGB to BGR for OpenCV. LaMa pads the input to a multiple of 8 pixels (reflect padding), converts to a float tensor in $[0, 1]$, and crops the padding from the output.

1.4 Mask experiments

The same pipeline derives several masks from the object mask in order to test sensitivity to mask precision, size and shape (Table 1).

1.5 Metrics and outputs

For every (sample, model, mask experiment) triple the benchmark computes PSNR, SSIM and LPIPS between the prediction and `bg`, on the masked region (primary) and on the full image (secondary check), together with inference time. Results are written to `metrics.csv` (one row per triple), `summary.csv` (mean and standard deviation per model and mask), side-by-side comparison images and bar plots.

Experiment	Source mask	Operation
sam_native	mask_sam	none (exact object mask)
sam_dilated10	mask_sam	dilation, radius 10
sam_dilated20	mask_sam	dilation, radius 20
sam_bbox	mask_sam	bounding box
sam_ellipse	mask_sam	enclosing ellipse
eff_native	mask_eff	none (object plus shadows/effects)

Table 1: Mask experiments run for every sample and every model.

2 Baseline Model: OpenCV Telea

2.1 Configuration

The baseline is the classical Telea method (`cv2.INPAINT_TELEA`), which propagates colour and texture from the mask boundary inward. It involves no semantic reasoning and no pretrained weights.

2.2 Training setup

None. Telea is a non-learned algorithm. Likewise, all learned methods in this project use pretrained weights and are evaluated without any fine-tuning.

2.3 Initial quantitative results



Figure 1: Example of Telea

Table 2 reports the mean metrics of Telea over CORNE-Val (218 samples) for each mask experiment. PSNR and SSIM are higher-is-better, LPIPS and time are lower-is-better. Region metrics are computed on the masked area only.

Mask	PSNR↑	SSIM↑	LPIPS↓	Time, s↓
sam_native	12.74 ± 4.01	0.603 ± 0.146	0.487 ± 0.111	0.014 ± 0.009
sam_dilated10	14.65 ± 4.48	0.664 ± 0.147	0.446 ± 0.099	0.018 ± 0.012
sam_dilated20	15.26 ± 4.75	0.671 ± 0.149	0.453 ± 0.095	0.021 ± 0.013
sam_bbox	14.45 ± 4.09	0.545 ± 0.196	0.608 ± 0.091	0.025 ± 0.021
sam_ellipse	12.49 ± 3.57	0.506 ± 0.162	0.605 ± 0.077	0.022 ± 0.019
eff_native	17.42 ± 5.40	0.722 ± 0.146	0.420 ± 0.099	0.023 ± 0.016

Table 2: Telea baseline, region metrics (mean ± std over CORNE-Val).

3 First Experiments with Pretrained LaMa

We then started testing the pretrained big-lama model (CNN-based, Fast Fourier Convolutions) with no fine-tuning, behind the same `inpaint(image, mask)` interface as Telea.

3.1 Current observations

Observation

With the current setup LaMa does not remove the object. Instead the object region is smeared: the result looks blurred, and the object is still recognisable.



Figure 2: Example of LaMa output

Quantitative LaMa results are not reported yet, because the current output is not yet a meaningful measurement of the method. We have not identified the cause and list the hypotheses below as things to test rather than conclusions.

3.2 Hypotheses and planned experiments

- **Input handling.** Verify that the TorchScript model expects exactly our tensor format (value range $[0, 1]$, mask with 1 = fill, mask binarisation threshold) and that reflect-padding the mask does not mark padded pixels as holes.
- **Resolution.** CORNE-Val images are large (up to about 1500 px wide), whereas LaMa is typically run at around 512 px. Test downscaling or tiling.
- **Mask.** Try to change the mask or preprocess the photo.

4 Problems and Observations

- Official big-lama weights on Hugging Face are not in a directly loadable format (see above); the TorchScript export is a third-party artefact and must be documented in the final report.
- ViTInpainter and FLUX.1-Fill are not integrated yet. The benchmark skips unavailable models, so it already runs with Telea and LaMa and will pick up the others once their wrappers exist.
- The shared interface makes it easy to swap models, but model-specific preprocessing (padding, colour order) must stay inside each wrapper.

5 Next Steps

Plan for week 3

- Diagnose the LaMa blurring using the experiments above and obtain a first valid LaMa result.
- Run the full benchmark for Telea and LaMa over all mask experiments.
- Integrate ViTInpainter and FLUX.1-Fill into the shared interface.
- Collect the first failure cases for the failure analysis.

6 Code Repository

The current project code is available at:
https://github.com/Kazualov/bye-bye-AI